Retrieval Augmented Generation & Agents

AI TRAINING

AI TRAINING

Content

- Retrieval Augmented Generation (RAG)
- Agents
- Agentic RAG
- Hands-on experience RAG and Agents

🚨 Why Do We Need RAG?

- 🤖 LLMs are great—but they:
  - 🔍 Hallucinate facts
  - 📅 Get stale over time
  - 🔐 Pose privacy risks when fine-tuned

➡️ We need a way to **inject real-world knowledge** at runtime.

🧩 What is RAG?

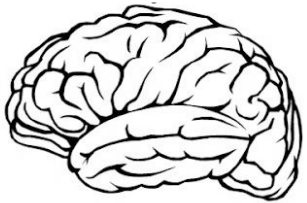
User

Data Source

🧩 What is RAG?

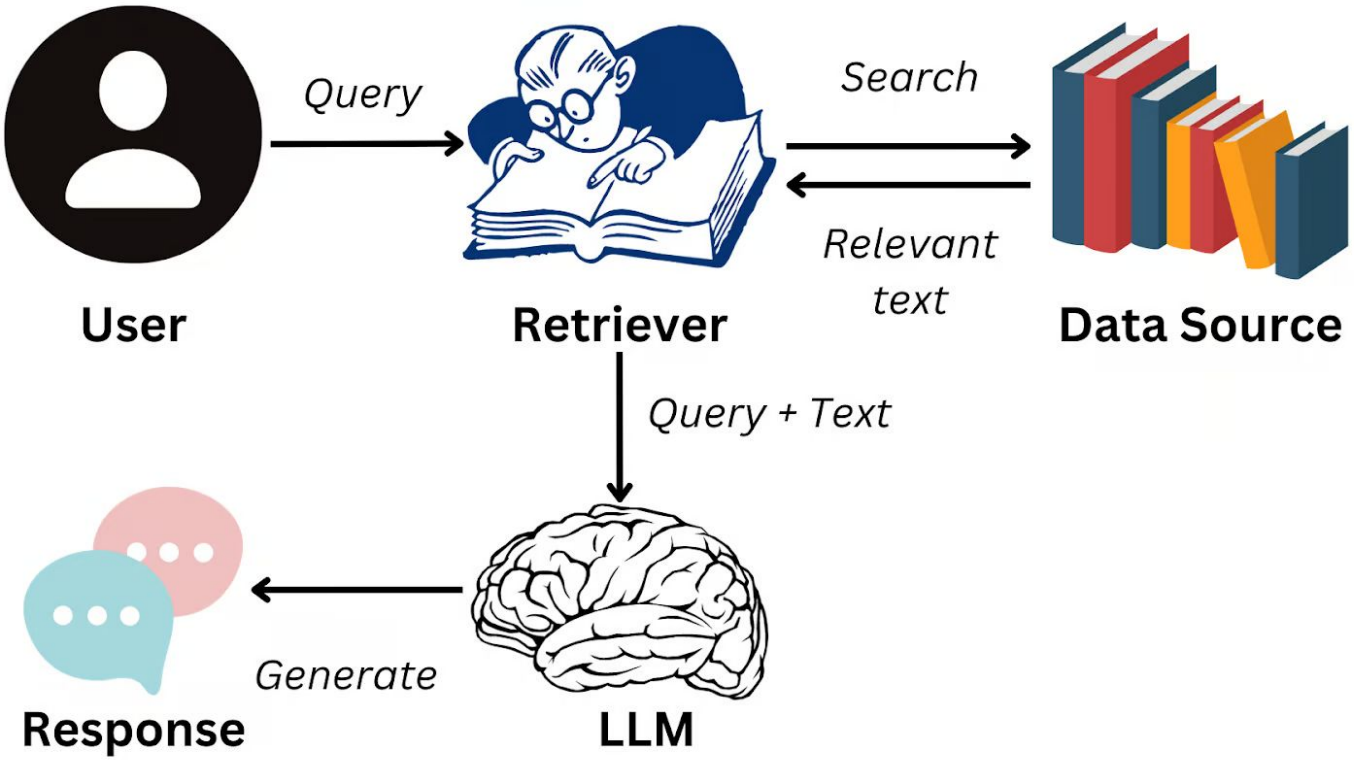
User

Data Source

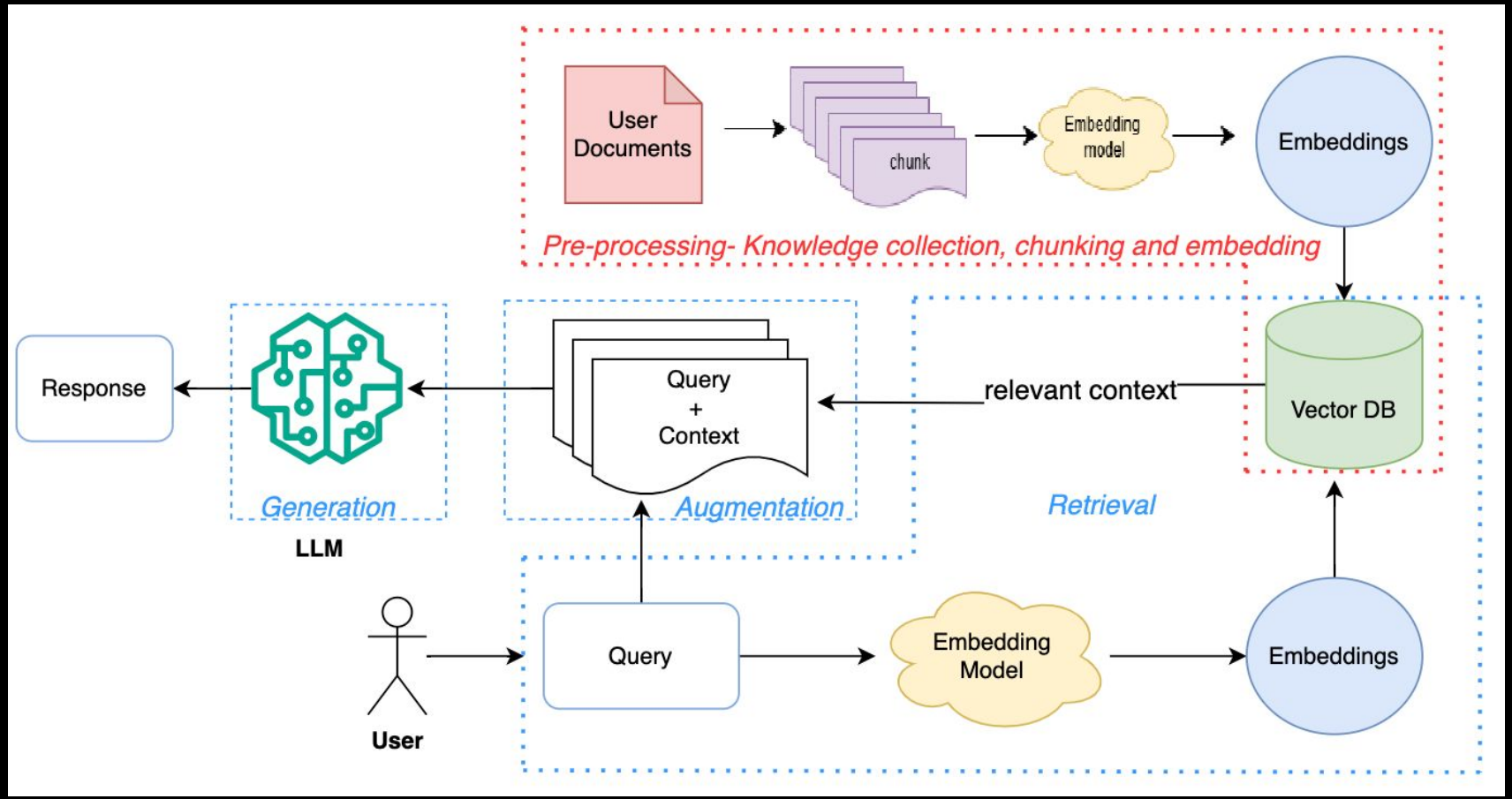


LLM

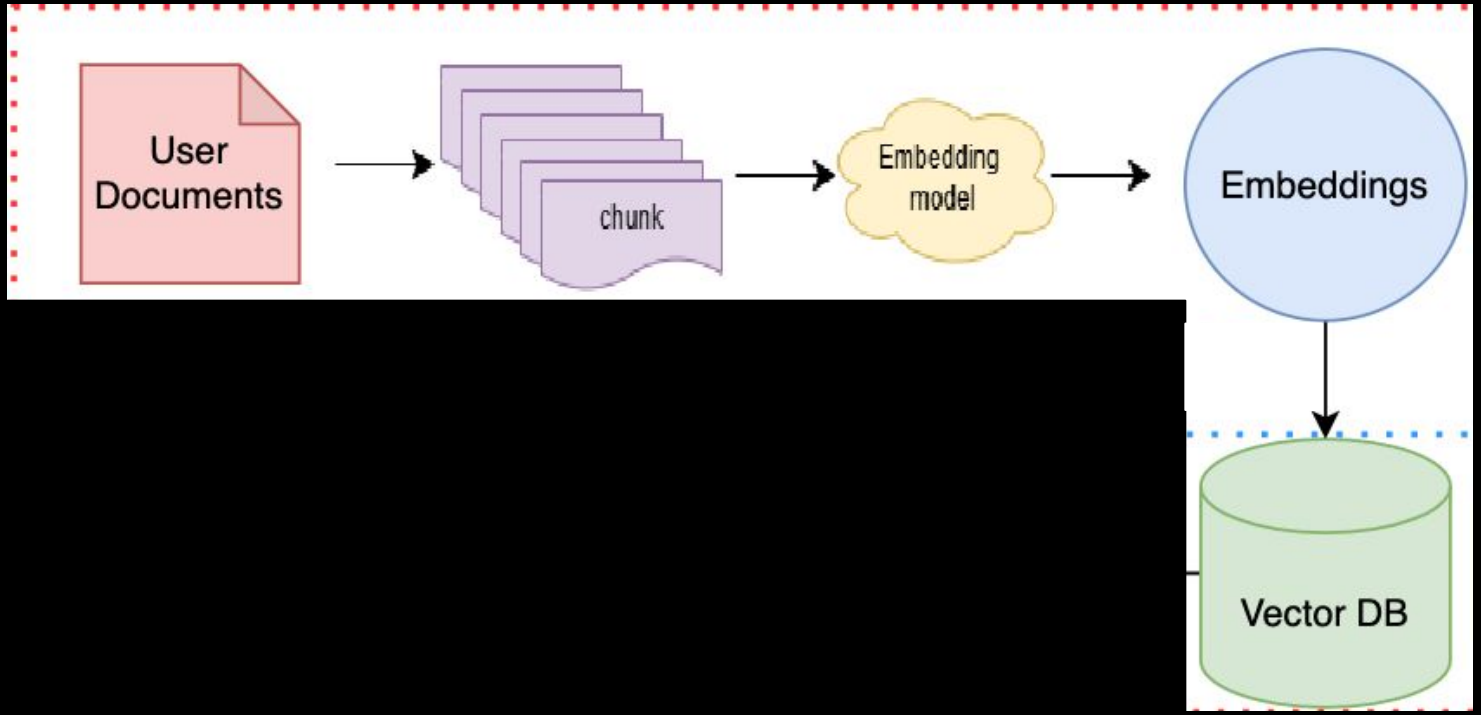
🧩 What is RAG?


Query
Search
Relevant text
User
Retriever
Data Source
Query + Text
Generate
Response
LLM

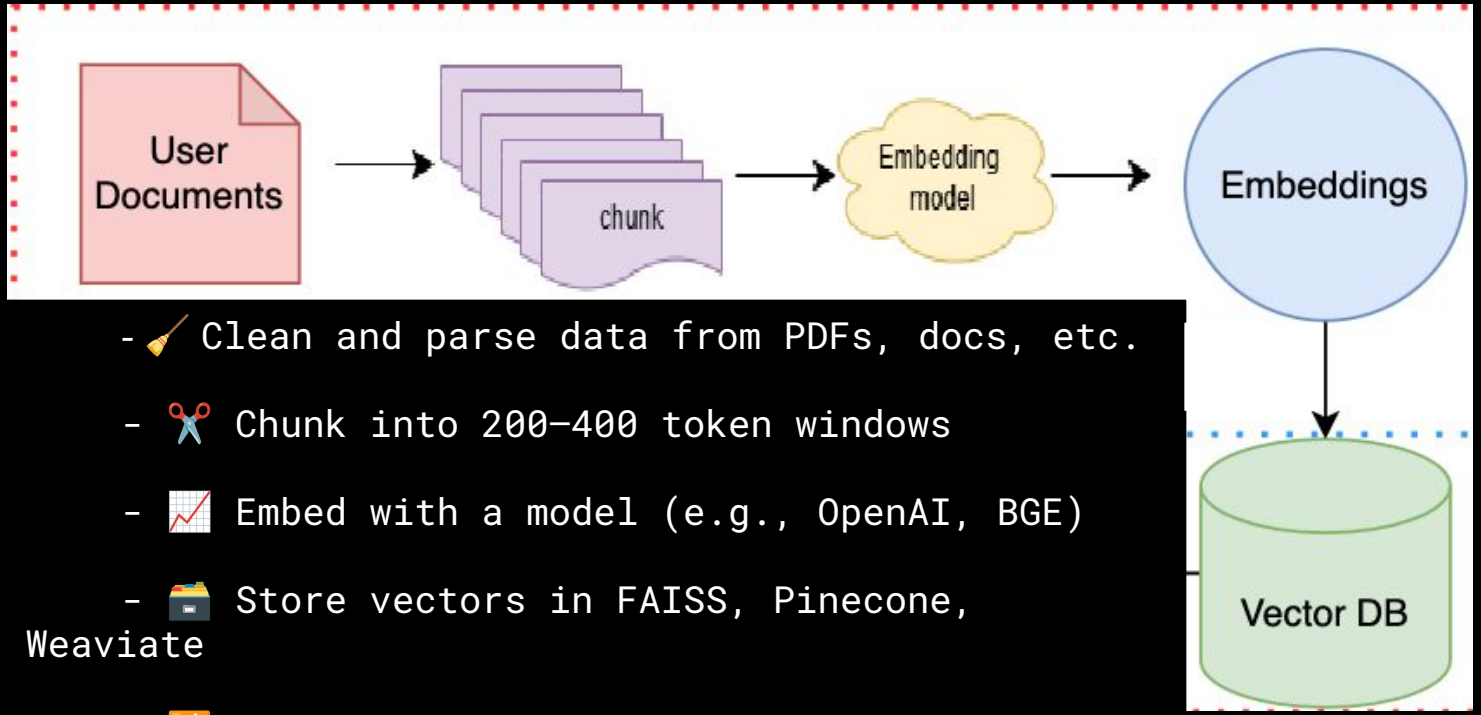
RAG Architecture



📄 Ingestion & documents embedding

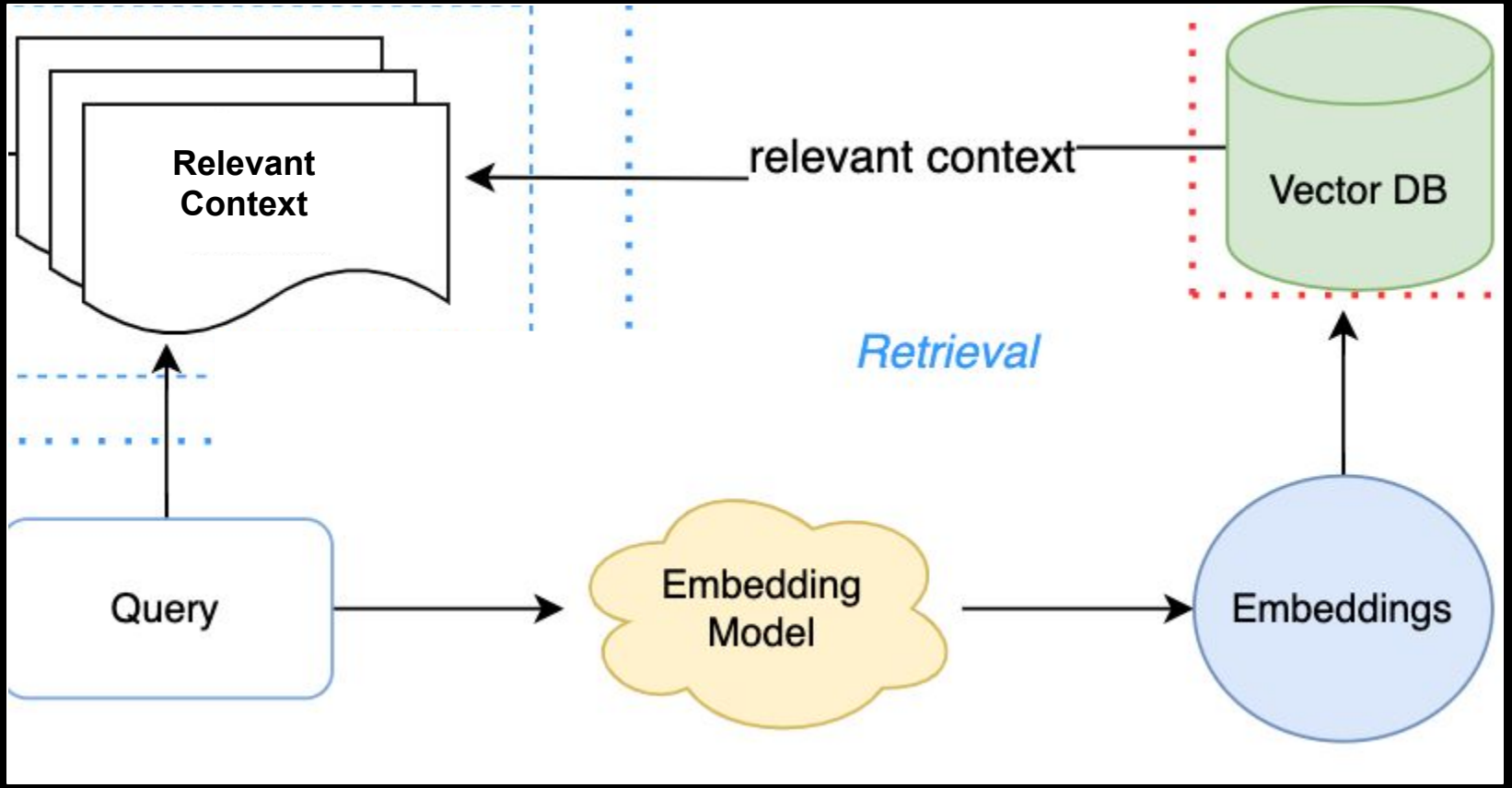


📄 Ingestion & documents embedding

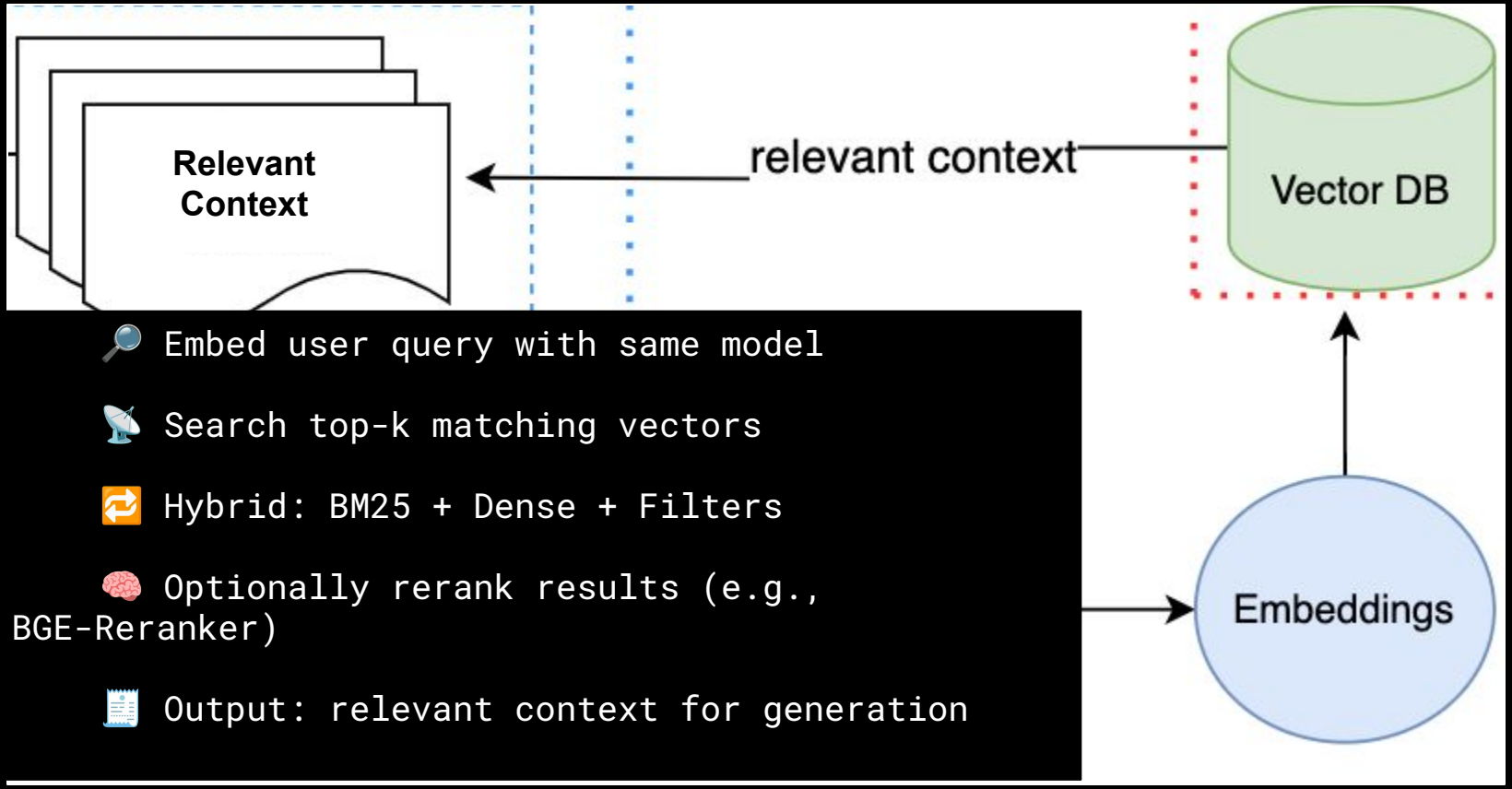


- 🧹 Clean and parse data from PDFs, docs, etc.
- ✂️ Chunk into 200–400 token windows
- 📈 Embed with a model (e.g., OpenAI, BGE)
- 🗃️ Store vectors in FAISS, Pinecone, Weaviate
- 🔁 Schedule regular refresh (daily or streaming)

🎯 Stage 2: Retrieval



🎯 Stage 2: Retrieval



🔎 Embed user query with same model

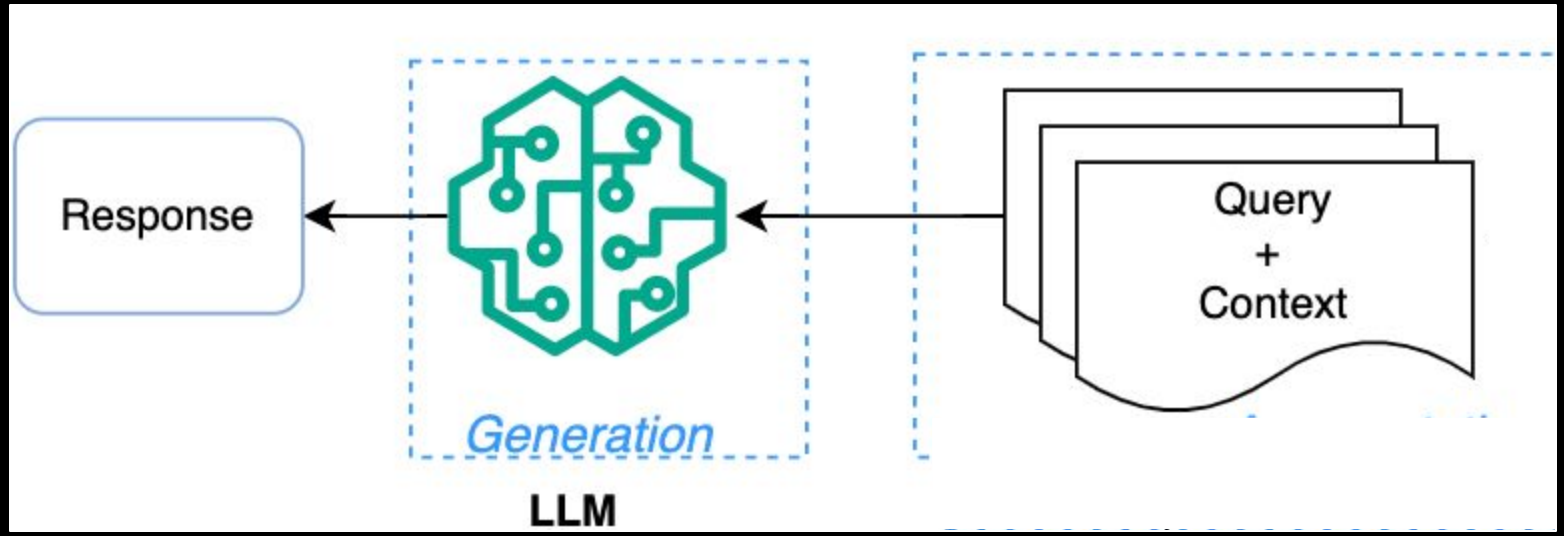
📡 Search top-k matching vectors

🔁 Hybrid: BM25 + Dense + Filters

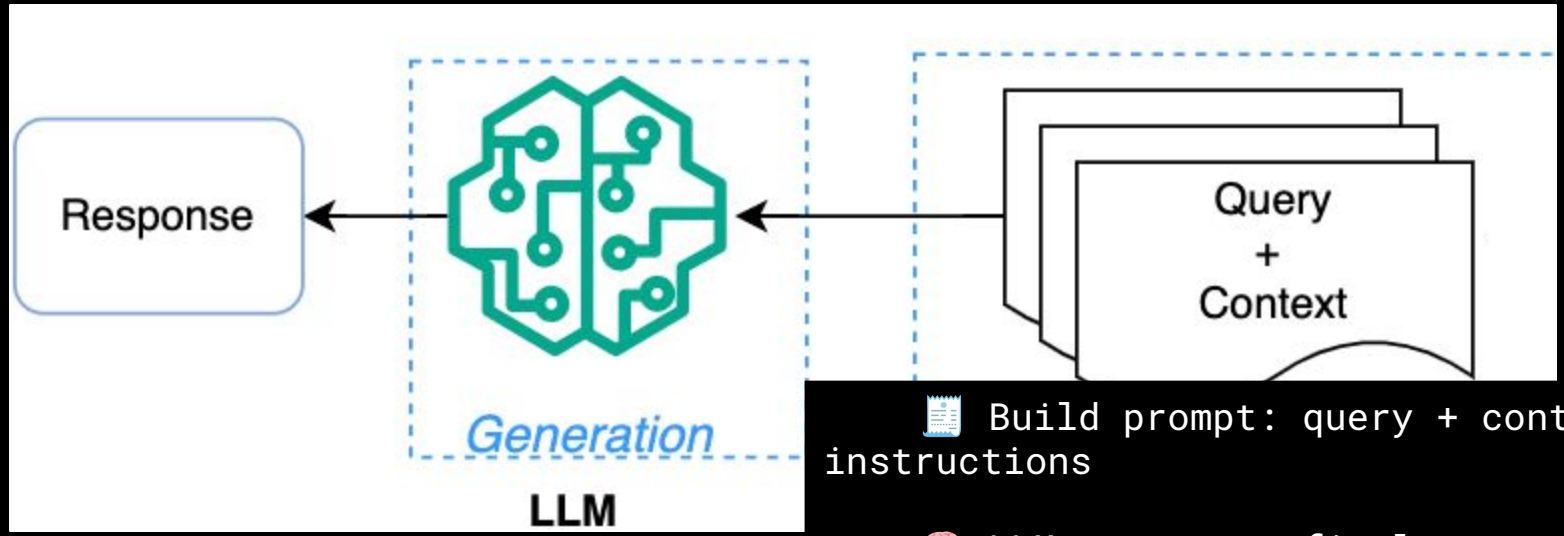
🧠 Optionally rerank results (e.g., BGE-Reranker)

🧾 Output: relevant context for generation

🧠 Stage 3: Generation



🧠 Stage 3: Generation



🗒️ Build prompt: query + context + instructions

🧠 LLM generates final answer

🔍 Model cites chunks with [1], [2], etc.

🔐 Apply guardrails (toxicity, PII, fallback)

✅ Return to user with sources

⚠️Challenges with Naive RAG

AI TRAINING

🔍 Bad Retrieval

📉 Low Recall

Fails to retrieve all relevant chunks

LLM lacks enough evidence to answer accurately

🎯 Low Precision

Retrieves irrelevant or loosely related chunks

- 🤯 Hallucinations
- 🌀 “Lost in the middle” — key info gets buried in clutter

⚠️Challenges with Naive RAG

AI TRAINING

💬 Bad Response Generation

🧪 Hallucination

LLM fabricates facts not supported by the retrieved content

❌ Irrelevance

Response doesn’t answer the question or misinterprets intent

☣️ Toxicity / Bias

Generates harmful, offensive, or biased output — a safety and trust issue

📊 Evaluation Overview

AI TRAINING

🔍 Retrieval

- 📈 Recall@k — Are the right documents retrieved?
- 🔁 MRR — How early is the first relevant chunk?
- 🧮 nDCG — How well-ranked are all relevant results?

🧠 Generation

- 🗣️ Fluency — Is the response grammatically natural?
- 🧷 Faithfulness — Are all claims backed by retrieved text?
- 📏 ROUGE — Overlap with reference answers (if available)

Key Takeaways

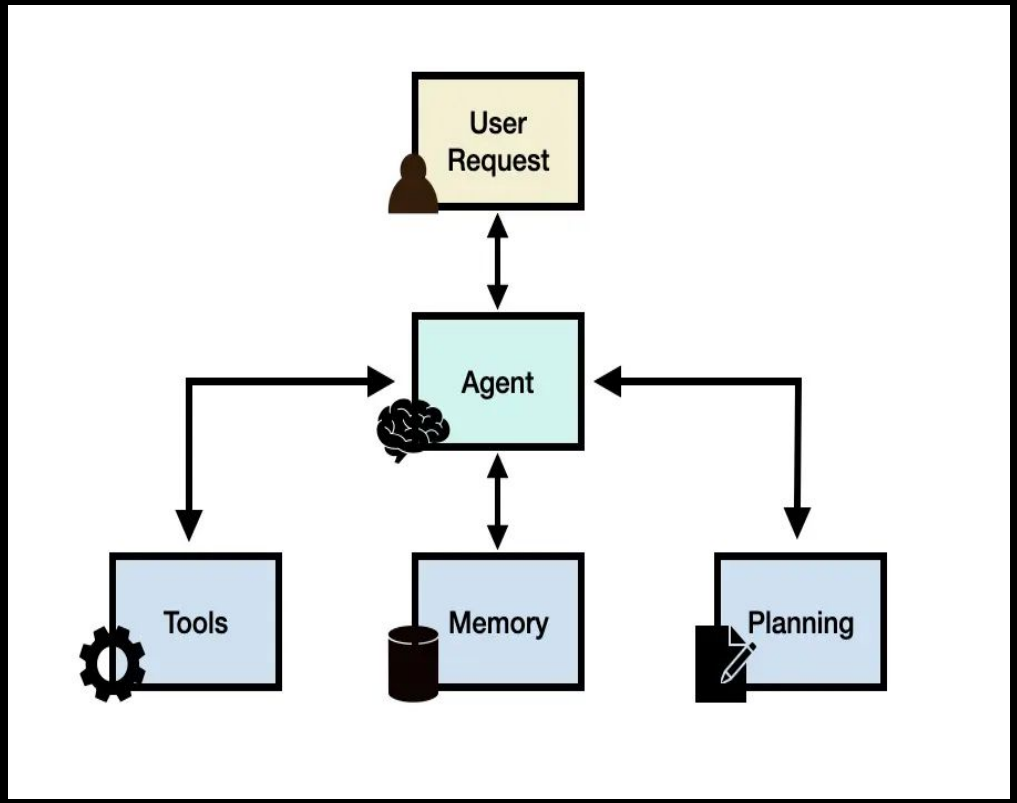
AI TRAINING

- ✅ Retrieval quality matters more than model size
- ✅ Evaluate both retriever and generator
- ✅ Add rerankers and filters early
- ✅ Agentic and multimodal RAG are next
- ✅ Always monitor faithfulness and safety

🧪 Next: Hands-on lab with LangChain + RAGAS + Giskard

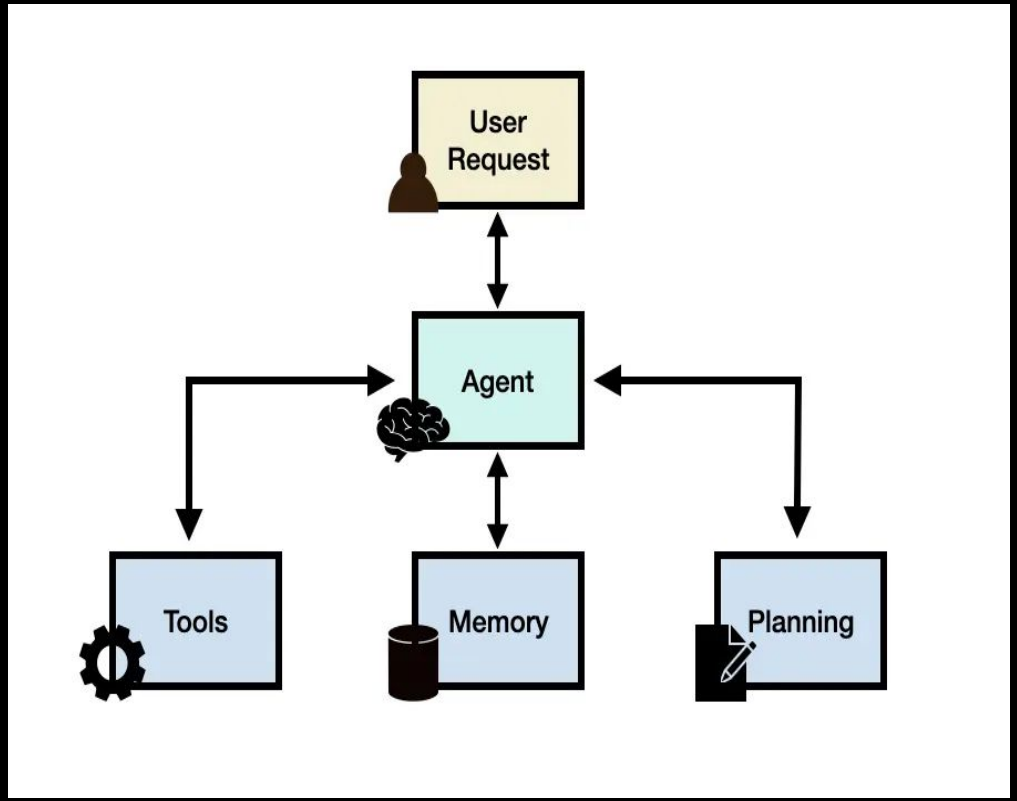
Agents

🧠 What Is an LLM Agent?



An LLM Agent is more than just a prompt → response engine.

🧠 What Is an LLM Agent?



An LLM Agent is more than just a prompt → response engine.

- It can:

  - 🗺️ Plan actions
  - 💡 Reason through tasks
  - 🛠️ Use tools (like calculators, APIs, retrievers)
  - 🧠 Maintain memory

- It acts with **autonomy** to reach a goal.

Example Task: Beyond Prompting

"Summarize these 3 PDFs, verify a fact online, and write a tweet about the result."

Example Task: Beyond Prompting

"Summarize these 3 PDFs, verify a fact online, and write a tweet about the result."

❌ Traditional LLM Approach

🧾 Needs all content + instructions packed into a single prompt

🧠 No intermediate reasoning, no tool use

🎲 Prone to:

- Hallucinations
- Missed step
- Token overload

Example Task: Beyond Prompting

"Summarize these 3 PDFs, verify a fact online, and write a tweet about the result."

❌ Traditional LLM Approach

🧾 Needs all content + instructions packed into a single prompt

🧠 No intermediate reasoning, no tool use

🎲 Prone to:

- Hallucinations
- Missed step
- Token overload

✅ LLM Agent Approach

📥 Download & Parse the 3 PDFs

🗺️ Plan sub-tasks: summarization → fact-check → tweet

🔍 Search the web or call an API to verify facts

🧠 Store intermediate results (in scratchpad or memory)

✍️ Compose the final tweet, citing accurate info

✅ Return a clean, fact-checked result

🧩 Core Components of an LLM Agent

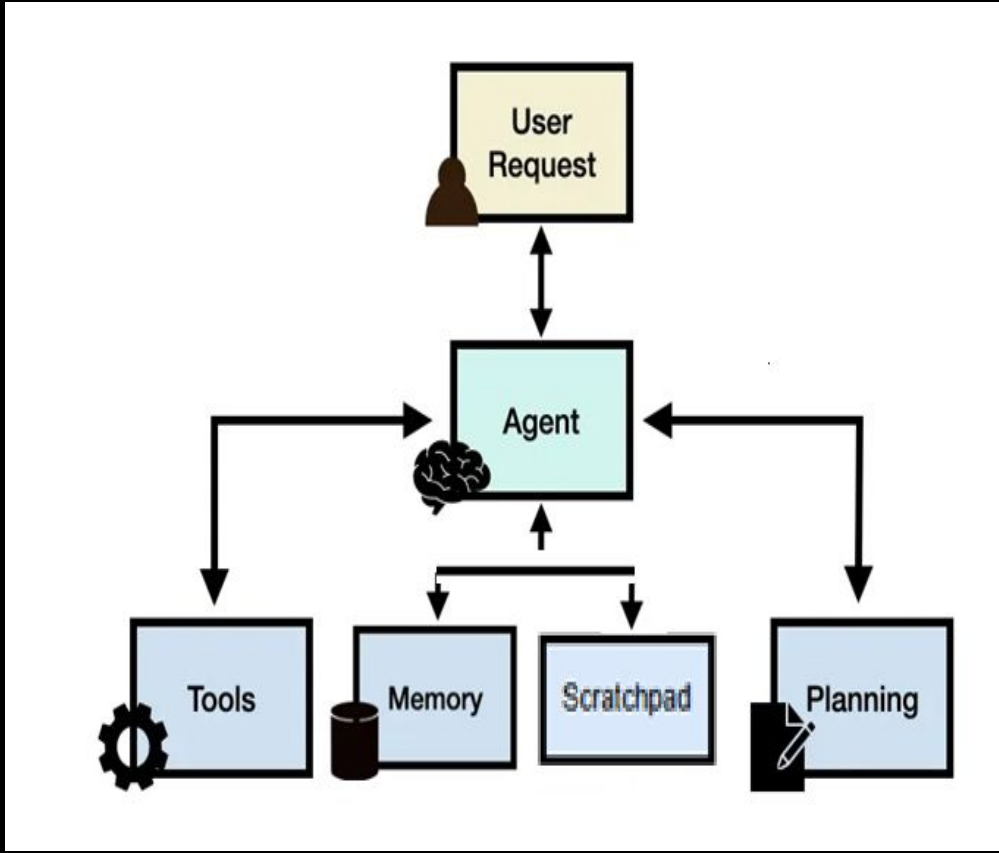
🧠 **LLM Controller**: the reasoning "brain"

- 🗺️ **Planner**: decides what steps to take

- 🔌 **Tools**

- 🗃️ **Memory:** stores past actions, context, or documents

- ✍️ **Scratchpad** holds intermediate thoughts or data



🔄 Agent Workflow

1. 🧾 User Input
2. 🧠 Task Planning
3. 🔍 Information Lookup
4. ⚙️ Tool Execution
5. 📝 Intermediate Reasoning
6. ✅ Final Answer
7. 🧠 Memory Update (optional)

🎯 Why Use Agents?

AI TRAINING

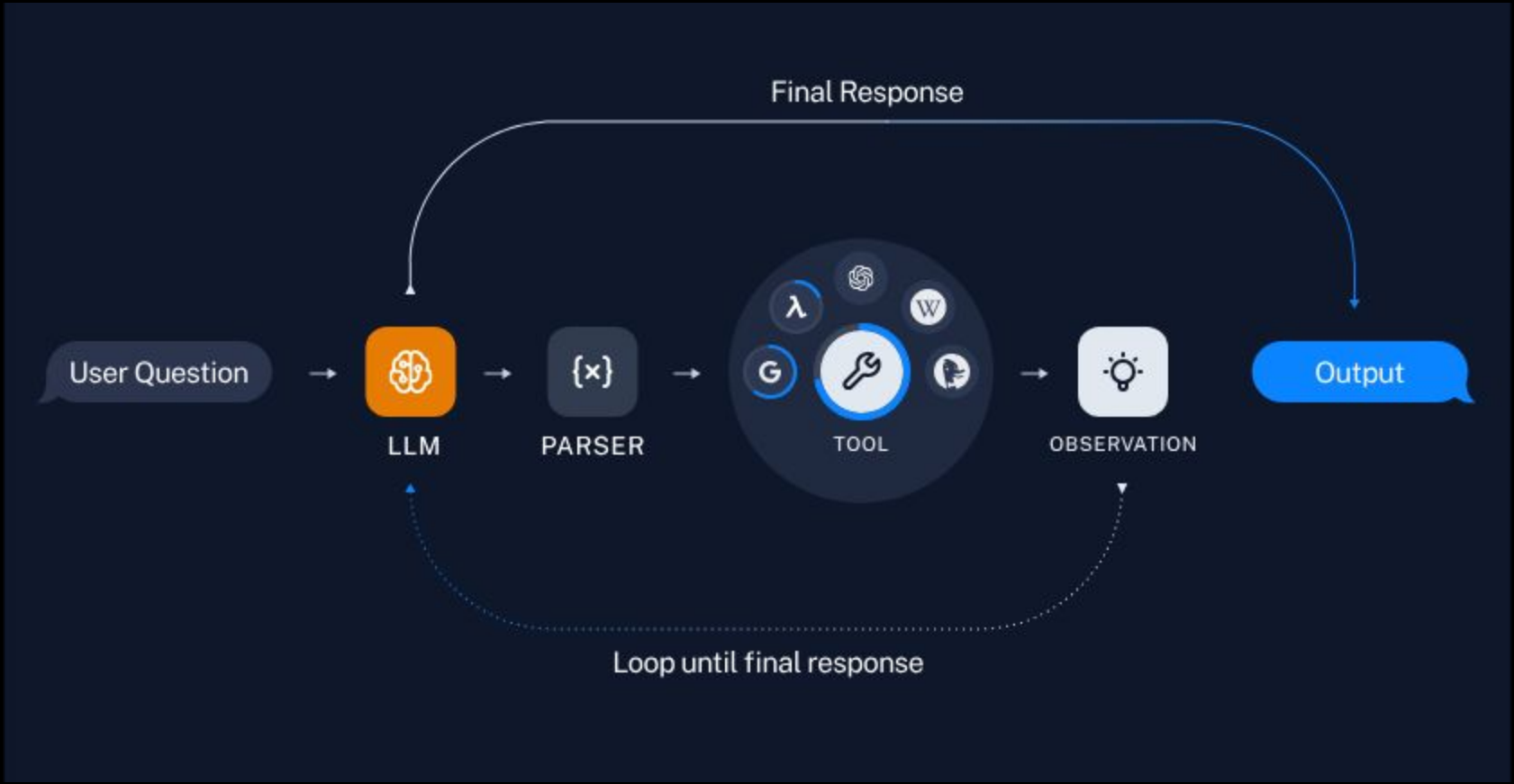
✅ **Autonomy**: Less human prompting, more reasoning

✅ **Multi-step reasoning**: Complex tasks handled in stages

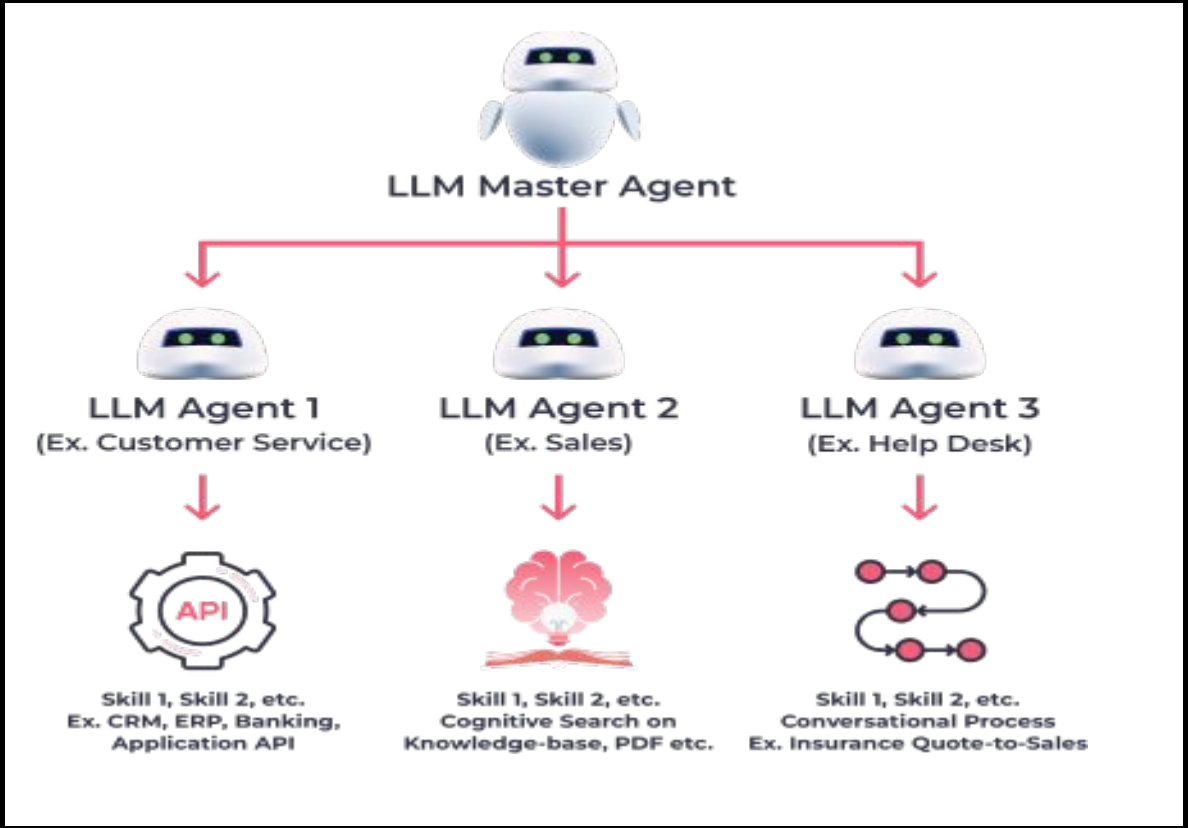
✅ **Tool use**: Go beyond what the LLM “knows”

✅ **Dynamic behavior**: Decide in real-time what to do next

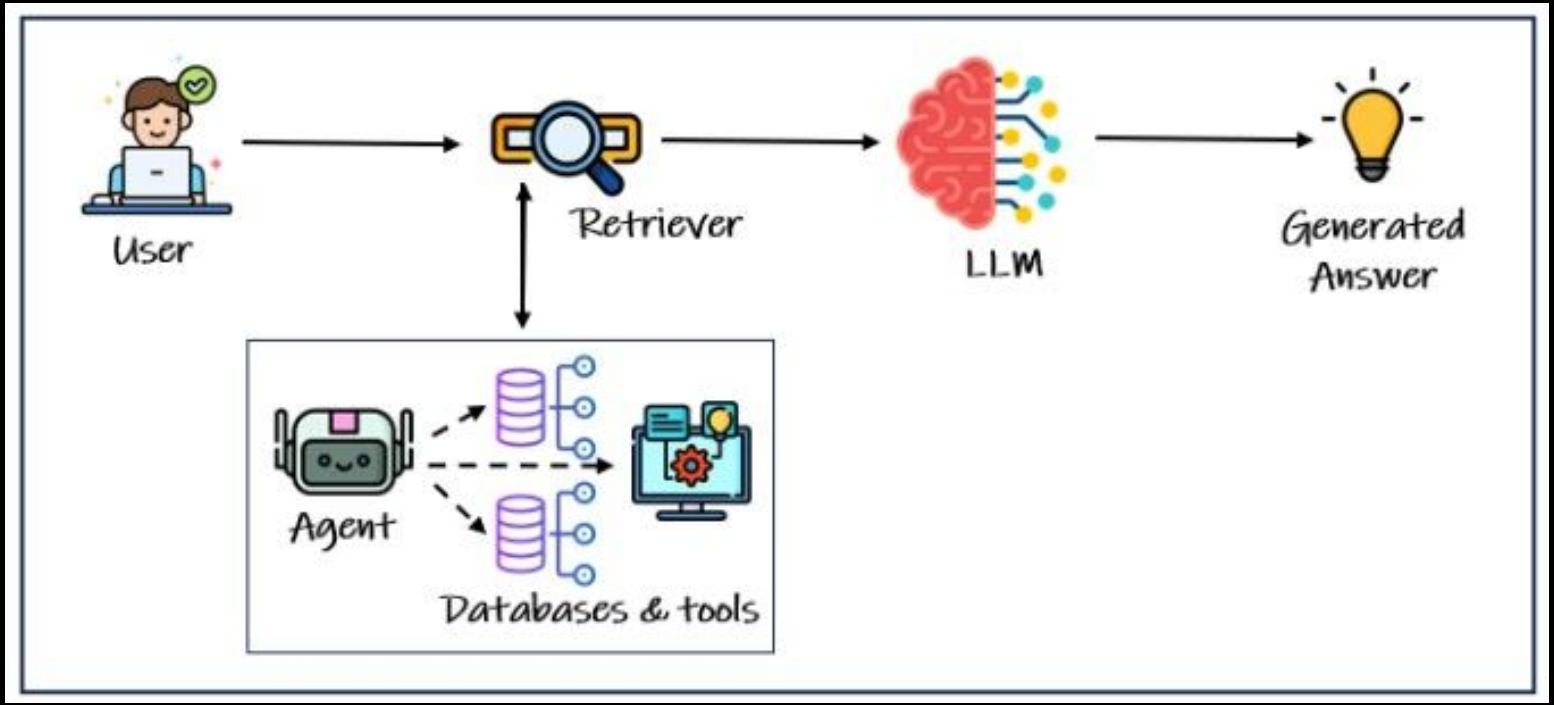
🏗️ Agent Architectures: Single-Agent



🏗️ Agent Architectures: Multi-Agent Systems

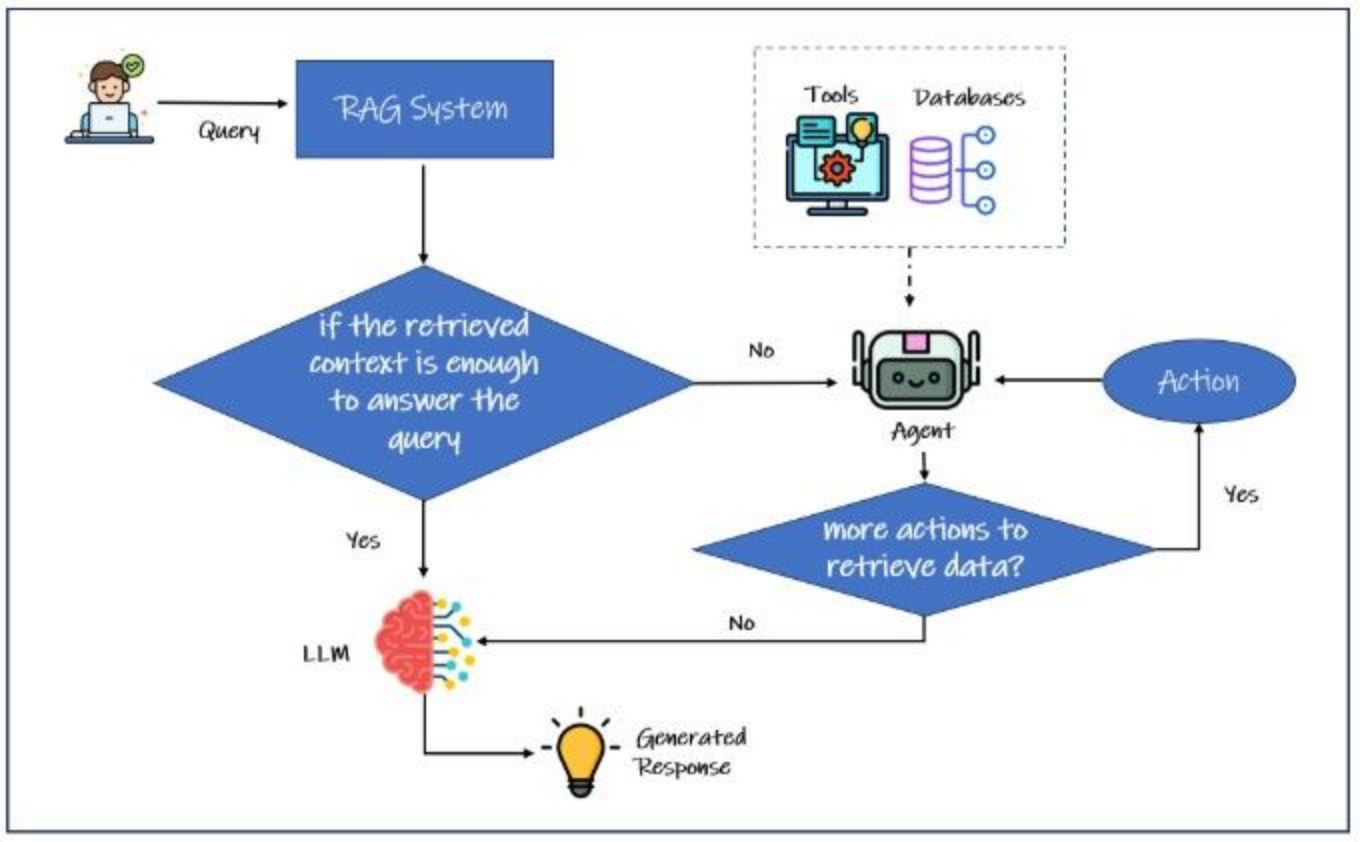


Agentic RAG



Source: https://www.deepchecks.com/glossary/agentic-rag/

Agentic RAG



Source: https://www.deepchecks.com/glossary/agentic-rag/

🧱 Challenges to Watch

AI TRAINING

🧪 Hallucinated tools or actions

🐢 High latency with many steps

🧩 Prompt orchestration complexity

📊 Harder evaluation pipelines

🔁 Maintaining long-term memory

Conclusion

- LLM Agents ≠ LLMs — they plan, reason, and act
- Combine tools, memory, and autonomy
- Agentic RAG enables smarter, context-aware QA
- Evaluation and traceability are key for production

🧪 Next: Hands-on lab

Langgraph

What is LangGraph?

Open-source orchestration framework developed by LangChain, designed to build, deploy, and manage complex generative AI agent workflows.

Key Features:

Stateful Graph Architecture: Utilizes nodes (tasks) and edges (transitions) to model workflows, maintaining context throughout.

Cyclic Computation: Supports loops within workflows, enabling agents to revisit steps as needed.

Scalability: Designed for production-level applications with fault-tolerant

Use Cases:Conversational agents, Complex task automation, Custom LLM-backed experiences

Core Components of LangGraph

AI TRAINING

Nodes: Represent individual computation steps or functions within the workflow.

Edges: Define the flow between nodes, supporting conditional logic and loops.

State: Maintains context or memory that is updated as the computation progresses, enabling dynamic decision-making.

Agents: Autonomous entities comprising an LLM, tools, and prompts, capable of performing complex, multi-step tasks.

Tools: External functionalities or APIs that agents can invoke to perform specific actions.

Memory: Supports both short-term (session-based) and long-term (persistent) memory, allowing agents to retain information across interactions.

Conclusion

- LLM Agents ≠ LLMs — they plan, reason, and act
- Combine tools, memory, and autonomy
- Agentic RAG enables smarter, context-aware QA
- Evaluation and traceability are key for production

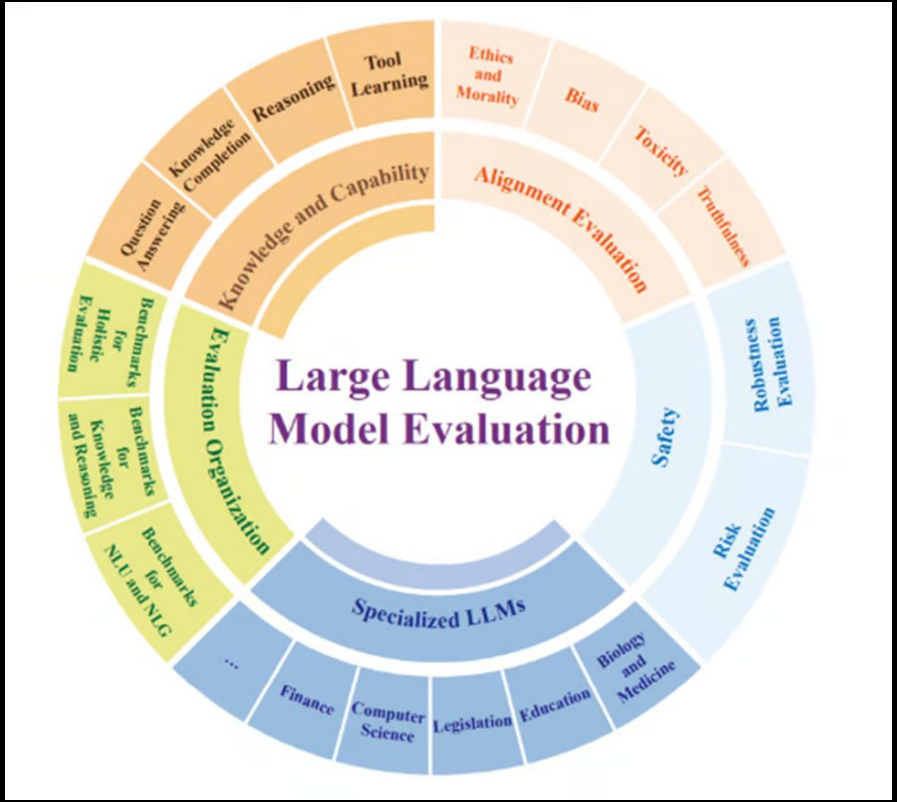
🧪 Next: Hands-on lab

RAG Evaluation

AI TRAINING

Unlocking the Power of LLMs

The Imperative to Evaluate LLMs



Giskard - Testing & Evaluating LLMs (RAG)

AI TRAINING

Gikard is an open-source framework developed to rigorously evaluate and secure LLMs and AI applications

✅ Detect Hidden Vulnerabilities: Automatically identifies issues such as hallucinations, harmful content generation, bias, and security risks

✅ Domain-Specific Evaluation: Uses both heuristic and LLM-assisted detectors to tailor tests to your business case.

✅ Continuous Quality Assurance: Easily integrates into CI/CD pipelines to monitor and alert on emerging risks before deployment.

Key Features:

✅ Giskard scan – Automatic Vunerability Detection

✅ RAGET – Purpose-built for RAG pipelines

Giskard LLM Scan

AI TRAINING

Goal: specialized tool designed to thoroughly analyze language model-based systems, identifying potential weaknesses and vulnerabilities (hallucination, harmful content generation, prompt injection, robustness, information disclosure, etc)

Key features: Tailor-Made for Business Models, Giskard is specifically designed for in-depth assessments of business-specific models, including chatbots, QA systems, RAG systems

Methodology: The scan utilizes a series of vulnerability-specific detectors to probe models

- Traditional Detectors: Utilize known techniques and heuristics to identify vulnerabilities.
- LLM Assisted Detectors: Employ another LLM model to probe and analyze the model under scrutiny.

Giskard's RAG Evaluation Toolkit Overview

AI TRAINING

Purpose & Scope:

Evaluates the performance of Retrieval-Augmented Generation (RAG) systems.

Complements vulnerability scans by providing end-to-end pipeline performance insights.

Key Components Evaluated:

- Retriever: Assesses document retrieval accuracy from your knowledge base.
- Generator: Measures the quality and relevance of LLM-generated responses.
- Optional Modules:
  - Rewriter: Optimizes user query reformulation.
  - Router: Directs queries based on intent.
  - Knowledge Base: Evaluates the coverage and pertinence of source documents.

Methodology:

- Uses a suite of tailored question types to stress-test each pipeline component.
- Generates detailed metrics to pinpoint areas for improvement